NovaCRM Onboarding Automation — Requirements & Workflow Analysis

Oct 8, 2026 · @Priyadharshini Pandian

Executive summary

NovaCRM's onboarding set-up moves from a manual, inbox-driven process to an event-driven agent system that goes from "deal closed" email to a live Rocketlane project and a shared Slack channel without a CS person touching it.

The three failures Priya described each get a structural fix rather than a reminder:

- **Wrong template:** the plan tier is confirmed by the AE on a voice call, and the agent maps tier to template deterministically (Enterprise = 30 days, Growth = 14 days).
- **Data marked done but not verified:** both templates end Data Migration with a customer sign-off on the migrated data, and the Go-Live Readiness Review confirms that verification before launch.
- **Escalations falling through the cracks:** a Rocketlane automation notifies the Project Manager when a task is 1 day overdue and the Project Owner at 4 days overdue.

The solution is built as four LangGraph workflows plus one ReAct agent for Slack channel creation:

1. **Intake workflow** — classifies the deal email (guardrail, reply detection, LLM verdict) and extracts the deal.
2. **Deal-details workflow** — emails the AE for any missing required fields, then continues.
3. **Tier-confirmation workflow** — phones the AE with a voice agent to confirm Enterprise or Growth, with an email fallback.
4. **Provisioning workflow** — creates the Rocketlane project from the matching template, then hands over to the Slack agent.

ReAct communication agent — runs inside the provisioning workflow and creates the shared Slack channel: the LLM chooses which Slack tool to call next and writes the welcome message, with a finalizer that guarantees the setup completes.

These run behind a Gmail → Pub/Sub → FastAPI → Cloud Tasks pipeline, plus native Rocketlane configuration (two templates and one overdue-escalation automation).

Current workflow (as-is)

Today every onboarding is started and tracked by hand, and the only trigger is an email to a shared inbox.

1. **Deal closes.** The AE emails the CS team inbox with the customer name and the Salesforce opportunity link.
2. **Triage.** Someone on the CS team reads the email and decides the plan tier, usually from memory or by opening Salesforce. The tier is not in the email.
3. **Project set-up.** CS creates an Asana project from the onboarding template: about 15 tasks across Kickoff, Data Migration, Configuration and Go-Live.
4. **Customer channel.** CS creates a shared Slack channel with the customer.
5. **Kickoff.** CS schedules the kickoff call.
6. **Delivery.** The CSM works through each phase checklist and updates Asana manually.
7. **Escalation.** A task blocked for more than 3 days should be escalated. In practice this is often missed.
8. **Close-out.** The CSM writes a handoff summary and moves the customer to the ongoing Support team.

Plan	CSM model	Onboarding length
Enterprise	Dedicated CSM	30 days
Growth	Pooled CSM	14 days

Actors: AE, CS team inbox, CS Manager (Priya), CSM (dedicated or pooled), customer admin, Support team.

Systems: Gmail, Salesforce, Asana (moving to Rocketlane in this design), Slack, calendar.

Pain points and root causes

Each problem comes from a step that depends on a person remembering something, so each fix removes that dependency instead of adding a reminder.

Problem	Impact	Root cause	Fix in this design
Wrong template applied	Growth customer gets a 30-day plan or Enterprise gets 14 days and a pooled CSM	Tier is missing from the AE email; a person guesses or looks it up	Agent calls the AE to confirm tier; tier → template mapping is code, not a choice

Problem	Impact	Root cause	Fix in this design
Data migration marked done but not verified	Bad data reaches go-live; "ugly escalations"	"Done" is a checkbox with no evidence or customer confirmation	Customer sign-off on migrated data; Go-Live Readiness Review confirms migration verification
Blocked tasks not escalated	Delays found late; customer frustration	Escalation relies on someone noticing a task is 3+ days stuck	Rocketlane automation on overdue tasks: 1 day → Project Manager, 4 days → Project Owner
Slow, manual set-up	Hours to days between deal close and kickoff	Inbox triage, project, channel and invite all done by hand	Event-driven agents create project and Slack channel within minutes of tier confirmation
No single source of truth	Status scattered across inbox, Asana and Slack	Data re-typed between systems	Rocketlane project carries tier, Salesforce link, AE, contact email and Slack URL as project fields

Requirements

The system must turn a deal email into a correctly tiered Rocketlane project and a Slack channel, with no silent failures.

Functional

ID	Requirement	Component
FR-1	Watch the CS Gmail inbox and react to new emails in near real time	Gmail push → Pub/Sub → FastAPI webhook
FR-2	Tell new deal emails apart from AE replies and other mail	Intake workflow: `guardrail`, `detect_escalation_reply`

ID	Requirement	Component
FR-3	Extract customer name, customer contact email, AE name, AE phone, Salesforce opportunity link and any notes	Intake workflow: `email_analyser`, `extract_deal`
FR-4	Phone the AE with a Voice AI agent and get a verbal Enterprise / Growth answer	Tier-confirmation workflow (Vapi voice agent)
FR-5	If the call fails or the answer is unclear, email the AE (or accept an operator override) and wait	`request_ae_reply`, `await_ae_reply`
FR-6	Create the Rocketlane project from the matching template with the correct start date and project fields	`create_rocketlane_project`
FR-7	Create a shared Slack channel with a tier-specific name, topic and welcome message, and invite the customer contact	Communication agent (ReAct, Slack tools)
FR-8	Notify the Project Manager when a task is 1 day overdue and the Project Owner at 4 days	Rocketlane automation
FR-9	Block Go-Live until the customer has signed off migrated data	Rocketlane templates (already built)
FR-10	If the deal email lacks a required field, email the AE for exactly the missing fields, wait for the reply, and continue only once every required field is valid	Deal-details workflow

Non-functional (infrastructure)

ID	Requirement	How it is met
NFR-1	Acknowledge Gmail push within 2 s so Pub/Sub does not redeliver	FastAPI returns 200 and hands off to Cloud Tasks
NFR-2	Process each email exactly once	Redis idempotency key, 24 h TTL; idempotent Rocketlane and Slack writes

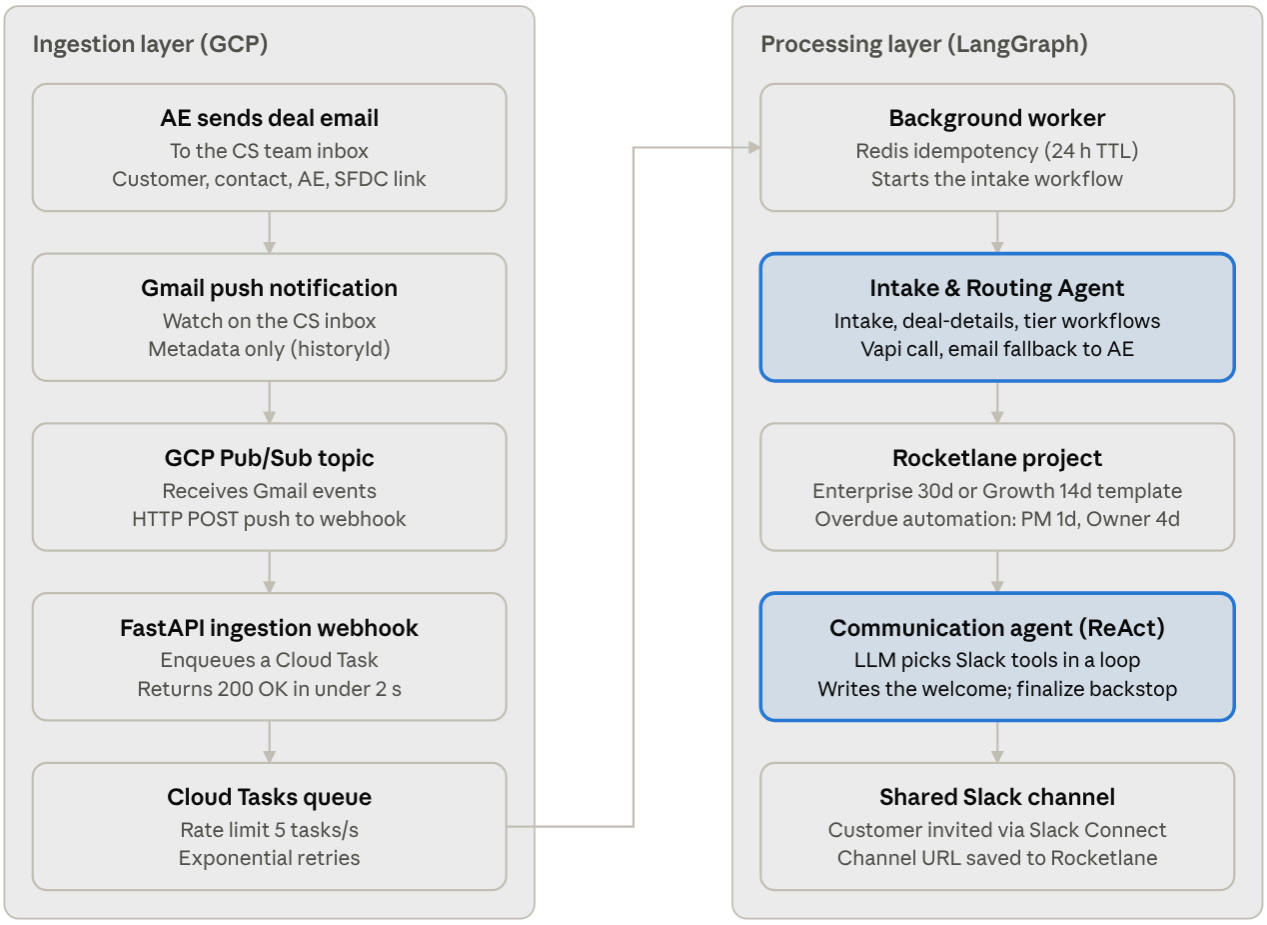
ID	Requirement	How it is met
NFR-3	Survive provider outages	Cloud Tasks exponential retries; call retry loop; durable LangGraph checkpoints
NFR-4	Protect against API rate limits	Cloud Tasks queue capped at 5 tasks/s
NFR-5	Keep PII and untrusted email text away from tool calls	Guardrail node (PII redaction, sender gate, injection and phishing checks), `fact_check_validator`
NFR-6	Respect the AE's working hours	Defer the call outside working hours
NFR-7	Every run is traceable	Structured logs and LangGraph state per Gmail thread

Proposed agentic workflow (to-be)

An AE email now starts the whole onboarding set-up; people only step in when the AE cannot confirm the tier on the phone.

End-to-end flow

One AE email flows through GCP ingestion to two agents



high-level architecture · ingestion to agents

AE replies to a tier request travel the same path; the worker classifies them as replies and resumes the paused graph.

1. **AE sends the deal email** to the CS inbox, as today. Nothing changes for the AE.
2. **Gmail push** sends a metadata-only notification (mailbox + `historyId`) to a GCP Pub/Sub topic.
3. **Ingestion webhook** (FastAPI) receives the Pub/Sub push, enqueues a Cloud Task and returns HTTP 200 in under 2 s.
4. **Cloud Tasks queue** delivers work at up to 5 tasks/s with exponential retries.
5. **Background worker** checks Redis for `processed_msg:{historyId}` (24 h TTL), records the email once in Postgres and starts the intake workflow.
6. **LangGraph orchestrator** runs the workflows in turn: intake (classify and extract) → deal details (only when fields are missing) → tier confirmation (phone the AE) → provisioning (Rocketlane project).

7. **Communication agent** (a ReAct agent) creates the shared Slack channel, invites the team and the customer, and writes and posts the welcome message.
8. **Rocketlane** takes over delivery: the template drives tasks and gates, and the overdue automation handles escalation.

What changes for each person

Person	Before	After
AE	Sends email	Sends the same email, then answers a short call (or one reply email) to confirm the tier
CS Manager / CS Lead	Triages inbox, builds project and channel	Is Project Owner; is notified only on 4-day overdue tasks or tier fallbacks
CSM	Sets up tools, chases tasks	Is assigned as Project Manager; starts at the kickoff task
Customer	Waits for a channel and invite	Gets the Slack invite and welcome message within minutes of tier confirmation

Agent design

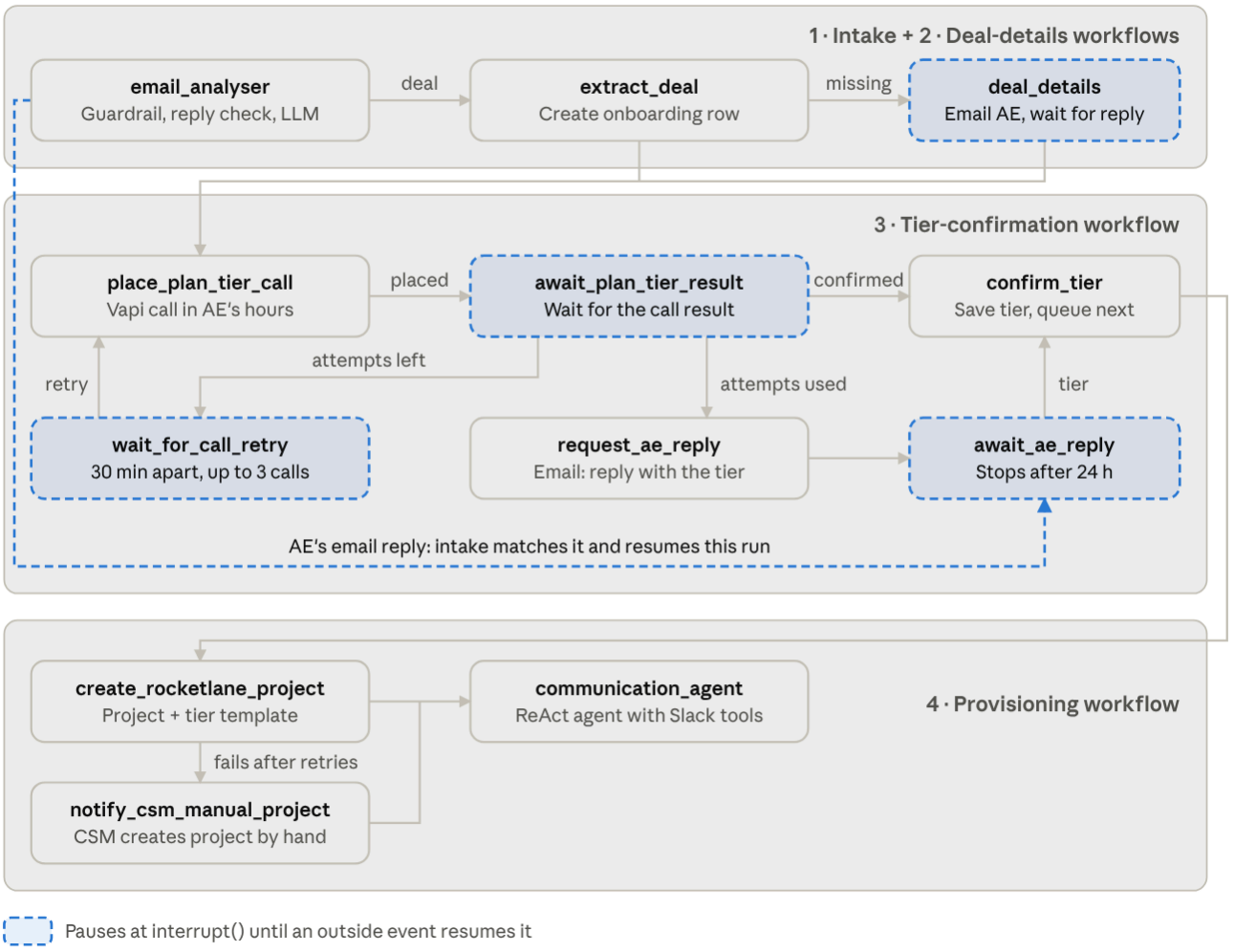
The work is split into four LangGraph workflows (intake, deal details, tier confirmation, provisioning) plus one ReAct agent for Slack channel creation, each workflow on its own thread with a Postgres checkpointer. A run can pause for hours waiting for the AE and resume exactly where it stopped.

Workflow	Thread	Job	Pauses for
`intake`	`intake:<inbound_email_id>`	Agent 1: classify the email; for a closed deal, extract details and create the onboarding	Never

Workflow	Thread	Job	Pauses for
`deal_details`	`details: <onboarding_id>`	Only when required fields are missing: email the AE for them, merge the reply, then hand off to tier confirmation	AE email reply
`tier_confirmation`	`tier: <onboarding_id>`	Agent 1: phone the AE with a Vapi voice agent; fall back to an email reply	Call result, call-retry timer, AE email reply
`provisioning`	`provision: <onboarding_id>`	Create the Rocketlane project, then run the Slack channel step	Never; a failure stops at the failing node

Each workflow hands off to the next by writing a `pending` row in `workflow_runs` before anything starts it. The orchestrator starts it straight away, and the workflow sweeper picks up any hand-off left behind by a crash.

Four workflows hand off through pending runs



four LangGraph workflows · hand-offs and pauses

An AE's email reply goes through the intake workflow first; it is classified as a reply and resumes the paused tier run on the same thread.

1. Intake workflow — classify the email

Node	What it does	Next
`fetch_email`	Composes the email text for the run	`guardrail`

Node	What it does	Next
`guardrail`	One node running four deterministic rules in order: redact PII → sender gate (must be a known AE) → prompt-injection check → phishing check. The first rule that blocks wins; no LLM sees an email that has not passed	Not an AE → `archive_skip` (no_deal); injection or phishing → `log_and_reject` (malicious); passed → `detect_escalation_reply`
`detect_escalation_reply`	Matches the email to an open tier request by Gmail thread or `[ONB-…]` subject token. No LLM	Reply → `handle_escalation_reply` → END, then the orchestrator resumes the tier run; otherwise `analyze_deal_status`
`analyze_deal_status`	LLM (Groq, Llama 3.3 70B, structured output) decides whether the email reports a closed deal	`fact_check_validator`
`fact_check_validator`	Rejects low-confidence verdicts and checks for closing phrases	Confirmed → `trigger_sales_system` (deal_closed); otherwise `archive_skip`
`extract_deal`	Extracts customer name, contact email and AE details; checks the required fields; creates the `onboardings` row; queues `tier:<id>` when complete, or `details:<id>` (section 2) when fields are missing	END

The guardrail is built on a reusable factory (`agents/commons/guardrail`): each rule is a plain function that returns findings, and the node writes one `guardrail_check` result that the graph routes on. A rule that raises (for example a database error during the sender lookup) fails the run at this node instead of letting an unchecked email through.

Phishing rules: one hard signal is enough (credential request, changed bank details, gift cards / wire / crypto, links to a bare IP, links with embedded credentials, punycode, or a display name showing a different address). Soft signals (verification lures, account threats, pressure, short links, "click here") count only in pairs.

2. Deal-details workflow — clarification email when fields are missing

If the deal email is missing the customer name or a valid customer contact email, the `deal_details` workflow (thread `details:<onboarding_id>`) asks the AE for them before the tier call. A complete email skips this workflow.

1. The LLM extracts the fields; code decides what is missing and treats any value not found in the email as missing.
2. `request_deal_details` emails the AE listing the missing fields and pauses.
3. The AE's reply is matched like a tier reply (Gmail thread or `[ONB-…]` token, sender must be that AE). `apply_deal_details` extracts and checks the new values.
4. Complete → the tier-confirmation workflow starts. Still missing → one follow-up email. No complete answer after two rounds or 24 h → the CS manager fills in the details.

3. Tier-confirmation workflow — confirm the plan with the AE

Node	What it does	Next
`place_plan_tier_call`	Looks up the AE's phone and timezone, checks the call window and places the Vapi call. One `voice_calls` row per attempt, so a replay never re-dials	Placed → `await_plan_tier_result`; outside hours → `wait_for_call_retry`; provider error → `schedule_call_retry`; no phone → `request_ae_reply`
`await_plan_tier_result`	`interrupt()` until the call result arrives	Confirmed → `confirm_tier`; attempts left → `schedule_call_retry`; attempts used → `request_ae_reply`
`schedule_call_retry` → `wait_for_call_retry`	Picks the next call time and pauses on a timer	`place_plan_tier_call`

Node	What it does	Next
`request_ae_reply`	Emails the AE "reply ENTERPRISE or GROWTH" with an `[ONB-…]` token, opens an `escalations` row and sets a 24 h stop time	`await_ae_reply`
`await_ae_reply`	`interrupt()` until the AE's reply or an operator resumes the same thread	`confirm_tier`
`confirm_tier`	Writes the plan tier on the onboarding; queues `provision:<id>` as pending	END

Call and reply rules

- Up to 3 call attempts, 30 minutes apart, only on weekdays inside the AE's working hours in the AE's own timezone.
- The tier is accepted only when the AE confirms the read-back with enough confidence. Not sure, voicemail, busy, wrong person or a stuck call all count as a failed attempt.
- The AE's email reply never goes through the LLM. Only the AE the request was sent to may answer, the reply must contain exactly one clear tier word, and an unclear reply gets a "one word please" nudge (at most twice).

AE reply path — from the escalation email to the Rocketlane project

When the calls fail, the AE's email reply is what unblocks the onboarding: the intake workflow recognises it as an answer, the paused tier run resumes, and provisioning continues exactly as after a confirmed call.

1. **AE replies** to the "reply ENTERPRISE or GROWTH" email. The reply arrives through the same Gmail notification path as any other email.
2. **Intake classifies it as a reply.** In `email_analyser`, `detect_escalation_reply` runs right after the guardrail (so only a known AE's clean email can count as a reply) and before any LLM step. It matches the email to an open `escalations` row by the Gmail thread ID of the email we sent, or by the `[ONB-…]` token in the subject. A match sets `kind = escalation_reply` and ends the intake run; no new onboarding is created.

3. **The reply is checked.** `TierReplyService` accepts it only from the AE the escalation was sent to. `parse_tier` needs exactly one clear tier word; an unclear reply gets a "one word please" nudge, at most twice.
4. **The tier run resumes.** The orchestrator resumes `tier:<onboarding_id>` with the tier. `await_ae_reply` wakes on the same thread, marks the escalation answered and passes the tier to `confirm_tier`. This works even if the run was already `stopped` after 24 h.
5. **`confirm_tier`** writes the plan tier on the onboarding and queues `provision:<onboarding_id>`.
6. **Provisioning continues.** `create_rocketlane_project` looks up the replied tier in `plan_tiers`, creates the project and imports that tier's template (Enterprise 30 days or Growth 14 days). `communication_agent` then creates the Slack channel.

The reply never goes through the LLM, so the tier that picks the Rocketlane template always comes from the AE's own words.

A reply to a deal-details request follows the same matching and sender check, but resumes `details:<onboarding_id>` instead (section 2).

4. Provisioning workflow — Rocketlane project and Slack channel

Node	What it does
`create_rocketlane_project`	Refuses to run without a confirmed tier. Reads the tier's row in `plan_tiers` (template ID, duration) and picks the CSM and CS manager; the project owner is the CS manager (the CSM when the tier has no manager). Writes a `creating` row, creates the project, then imports the tier's template. A retry skips a project that already exists. Temporary errors are retried; if creation still fails, the run goes to notify_csm_manual_project
`notify_csm_manual_project`	Rocketlane creation failed: notifies the CSM (CS manager in copy) to create the project manually, marks the onboarding for human review, and lets provisioning continue to the Slack channel. Detailed below
`communication_agent` (ReAct subgraph)	An LLM agent that uses six Slack tools to open the shared channel and writes the welcome message itself. Detailed below

When Rocketlane project creation fails

A Rocketlane outage or API error must not leave the customer waiting, so after automatic retries the CSM takes over project creation by hand.

1. `create_rocketlane_project` retries temporary errors. If the project still cannot be created (or the template import fails), it records the error on the `rocketlane_projects` row.
2. `notify_csm_manual_project` emails the CSM, with the CS manager in copy. The message gives the customer, the plan tier, the template to use (Enterprise 30 days or Growth 14 days), the start and go-live dates, the owner, and the error.
3. The onboarding is marked `needs_human_review` with the reason "Rocketlane project to be created manually".
4. Provisioning continues: the communication agent still opens the Slack channel, and the welcome message leaves out the project link.
5. The CSM creates the project from the right template and records its ID through the operator endpoint (`POST /onboardings/{id}/rocketlane-project`). The link is saved, the onboarding returns to `project_created`, and the CSM shares the link in the Slack channel.

If a partly created project exists (created, but the template import failed), the email says so, and the CSM imports the template into that project instead of creating a new one.

Communication agent (ReAct)

The communication agent is a bounded ReAct loop: the LLM decides which Slack tool to call next and writes the welcome message, while the tools do the work and a `finalize` node guarantees the setup is completed and recorded truthfully.

```
START — no Slack token ─▶ skip_communication ─▶ END
  └─▶ prepare ─▶ agent ─┬─ tool calls ─▶ tools ─▶ agent … (bounded loop)
      (facts brief)     └─ done / LLM down / step limit ─▶ finalize ─▶ END
```

Tool	What it does
`lookup_team_in_slack`	Finds the CSM, CS manager and AE in Slack by stored ID or work email
`create_slack_channel`	Creates the channel or reuses an existing one; on `name_taken` the model retries with the second name option
`set_channel_topic`	Sets the topic (the model may tighten wording, not facts)

Tool	What it does
`invite_internal_team`	Invites the team members found in Slack
`post_welcome_message`	Checks the LLM's draft, swaps `{{csm}}` / `{{manager}}` / `{{ae}}` for real mentions and posts it once
`invite_customer_to_channel`	Sends the Slack Connect invite to the contact on the onboarding record

Guardrails on the agent

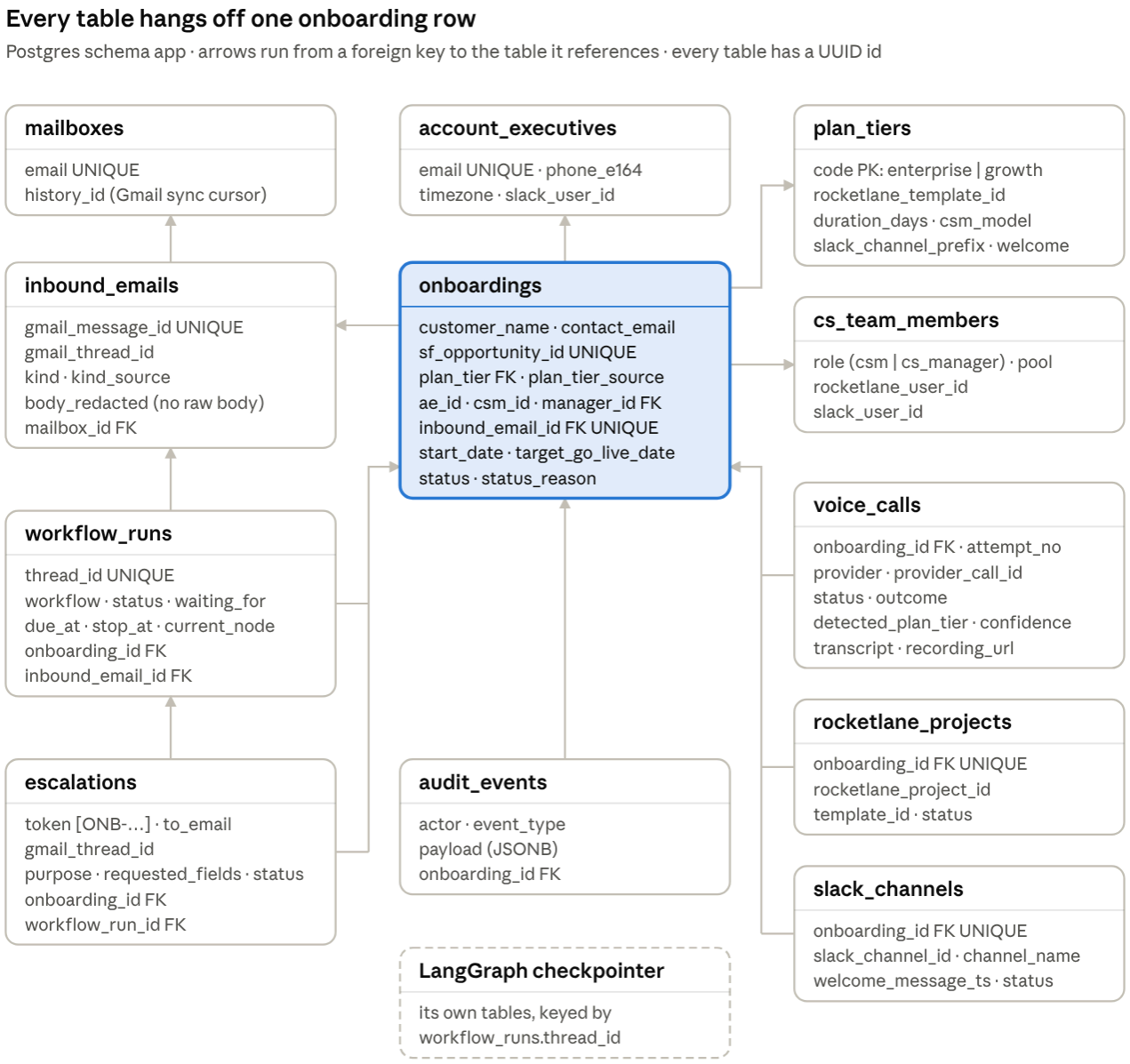
- The onboarding a tool acts on comes from graph state (`InjectedState`) and addresses come from the database, so the model cannot aim a tool at another onboarding or email address.
- The welcome draft is rejected, with reasons the model can act on, if it is too short or long, pings @channel / @here / @everyone, contains a raw Slack mention, or links anywhere except the project URL.
- Customer-supplied names sit inside `<facts>` and are declared data, not instructions.
- Slack calls are retried on temporary failures before the model sees them; results say "temporary" or "permanent" so the model knows whether to retry.
- The loop is capped at 14 LLM turns, the conversation is windowed to the last 24 messages, and it lives only inside the subgraph, so the provisioning checkpoint stays small.
- A Slack problem never fails the onboarding: any warning sends it to human review, and the Rocketlane project already exists.

The LLM never chooses a template: it only ever produces `enterprise` or `growth` , and the `plan_tiers` row decides the template, duration, CSM model, channel prefix and welcome text.

	Enterprise	Growth
Rocketlane template	Enterprise Onboarding	Growth Onboarding
Duration	30 days	14 days
CSM model	Dedicated CSM	Pooled CSM ("CS lead" in the channel topic)

Data model

All state lives in one Postgres database (schema `app`), centred on one `onboardings` row per deal. The LangGraph checkpointer keeps its own tables in the same database, keyed by `workflow_runs.thread_id`.



Postgres schema · 12 tables in schema app

Table	Holds	Key constraints
`mailboxes`	The watched Gmail inbox and its `history_id` sync cursor	`email` unique

Table	Holds	Key constraints
`inbound_emails`	Every email received: redacted body, `kind` (deal_closed / no_deal / escalation_reply / malicious), classification	Unique per (mailbox, Gmail message ID)
`account_executives`	AEs: email, phone, timezone, Slack ID	`email` unique
`plan_tiers`	Enterprise / Growth: Rocketlane template ID, duration, CSM model, Slack prefix, welcome template	`code` is the primary key
`cs_team_members`	CSMs and CS managers with Rocketlane and Slack IDs	`email` unique
`onboardings`	One row per deal: customer, AE, plan tier, CSM, manager, dates, status	`inbound_email_id` and `sf_opportunity_id` unique
`voice_calls`	Every call attempt: provider call ID, outcome, detected tier, confidence, transcript	Unique per (onboarding, attempt number)
`escalations`	Emails sent to the AE (tier request or missing details): token, Gmail thread, purpose, requested fields, answer	Unique per (onboarding, attempt number)
`workflow_runs`	One row per workflow run: thread ID, status, `waiting_for`, `due_at`, `stop_at`, current node	`thread_id` unique; one run per (onboarding, workflow)
`rocketlane_projects`	The project created: ID, template, owner, dates, status	One per onboarding
`slack_channels`	The Slack channel: ID, name, invite status, welcome message timestamp	One per onboarding
`audit_events`	Every decision and side effect, with actor and payload	Indexed by onboarding and time

The unique constraints carry the idempotency: one `inbound_emails` row per Gmail message, one `voice_calls` row per attempt, and one `rocketlane_projects` and `slack_channels` row per onboarding. A replayed node finds its row and skips the side effect.

Rocketlane configuration

Two templates and one automation live natively in Rocketlane; the workflow only picks the template and fills in the project fields.

Both project templates are already built in Rocketlane: **NovaCRM – Enterprise Onboarding (30 days)** and **NovaCRM – Growth Onboarding (14 days)**, each covering Kickoff, Data Migration, Configuration and Go-Live. The workflow only picks the template that matches the confirmed tier.

Overdue task escalation (Rocketlane automation, not an agent)

Rule	Trigger	Action
Escalation level 1	Task is overdue by 1 day and not completed	Notify the Project Manager (in-app + email) with task, assignee, due date and project link
Escalation level 2	Task is overdue by 4 days and not completed	Notify the Project Owner (CS Lead) with the same details

The automation is set at template level so every new project gets it automatically.

Assumptions

These assumptions shaped the design; any of them can be changed once confirmed.

Intake and email

1. AEs keep sending deal emails to one shared CS inbox, and the format is free text rather than a fixed form.
2. The email contains the customer name, customer contact email, AE name and Salesforce opportunity link. The AE's phone number and timezone come from the account_executives table, not from the email.
3. Plan tier is never in the email, so the AE call is always needed; the agent does not read the tier from Salesforce.

4. There are only two tiers, Enterprise and Growth. Any other answer is treated as "unclear" and goes to the human fallback.

Voice call

5. Vapi is the production voice provider for its developer control (per-call assistant config, function tools, server webhooks). The demo uses Bolna; both sit behind one send_call() function, so the graph does not change.
6. Calls are placed only on weekdays, within configured working hours in the AE's own timezone.
7. A verbal "Enterprise" or "Growth" from the AE is enough authority to start the project. No second approval is needed.

Rocketlane

8. Rocketlane replaces Asana as the onboarding system of record.
9. Both templates are already built in the Rocketlane UI; the agent creates projects from them via the Rocketlane API using the template ID.
10. Project start date = the day the tier is confirmed. Durations are calendar days from that date.
11. Rocketlane project owner = the CS manager (the CSM when the tier has no manager). Project Manager = the dedicated CSM (Enterprise) or a pooled CSM (Growth), picked from cs_team_members by tier (currently at random; round-robin or workload-based assignment is a follow-up).
12. The "blocked for more than 3 days" rule is implemented as the requested overdue-based escalation (1 day → Project Manager, 4 days → Project Owner), because due dates exist on every task and "blocked" is not tracked consistently today.
13. Template contents (tasks, subtasks, durations) are owned by the CS team in Rocketlane and are out of scope for this document.

Slack

14. NovaCRM's Slack plan supports Slack Connect, so customers can be invited to a shared channel by email.
15. Scheduling the kickoff call stays a template task for the CSM unless the agents also book it. The welcome message links to a booking page.

Clarifying questions and open items

These questions go to Karthik (karthikr@rocketlane.com); each answer would change a specific assumption above.

- [ ] Is there a standard format for the AE deal email, or should the agent handle free text? *(Assumption 1)*
- [ ] Where should the AE's phone number come from: the email signature, Salesforce, or an AE directory? *(Assumption 2)*
- [ ] If the AE says something other than Enterprise or Growth (for example a custom or pilot plan), should the agent stop or pick the nearest template? *(Assumption 4)*
- [ ] Should a CS Lead approve the tier, or is the AE's verbal confirmation final? *(Assumption 7)*
- [ ] Are onboarding durations in calendar days or working days? *(Assumption 10)*
- [ ] How is a CSM chosen from the Growth pool: round-robin, workload, or region? *(Assumption 11)*
- [ ] Should "blocked" also be tracked as a task status, with its own escalation alongside the overdue rules? *(Assumption 12)*
- [ ] Does NovaCRM's Slack plan support Slack Connect, and who should be invited from NovaCRM's side? *(Assumption 14)*
- [ ] Should the agents book the kickoff call directly in the CSM's calendar? *(Assumption 15)*
- [ ] Should the handoff summary at the end of onboarding be generated by an agent from Rocketlane and Slack activity? *(Future phase)*
- [ ] Which deal-email fields are blocking (currently: customer name and contact email) and which are only requested (contact name, Salesforce link)? *(Deal-details workflow)*